

ENSAM Meknès

École Nationale Supérieure
d'Arts et Métiers

ENSAM-Agent-OS

Système d'Exploitation Agentique
pour l'Université

Architecture, Gouvernance et Observabilité

Rapport de Projet de Recherche Appliquée

Projet 7 — Intégration Globale

Réalisé par :

BOUTRID Mourad

Encadré par :

Prof. Hajji Tarik

Module :

Représentation et Résolution
de Problèmes en IA

4^e année - IATD-SI

Année universitaire 2025–2026

Résumé

Résumé en français

Cet article présente **ENSAM-Agent-OS**, un système d'exploitation agentique local conçu pour le contexte universitaire de l'ENSAM Meknès. Face aux limites des solutions monolithiques à base de grands modèles de langage (LLM), nous proposons une architecture modulaire intégrant six agents spécialisés, une mémoire tri-couche (éphémère, sémantique, graphe de connaissances), un pipeline de génération augmentée par récupération (RAG), un registre d'outils conforme au *Model Context Protocol* (MCP) et un mécanisme de débat inter-agents.

L'innovation centrale réside dans la **gouvernance** du système : routage hybride local/cloud sensible aux données personnelles (PII), traçabilité complète des décisions, suivi des coûts en temps réel, conformité GDPR avec droit à l'oubli, et tableau de bord d'observabilité.

Notre prototype, évalué sur cinq scénarios d'usage réels (tutorat, recherche, administration, charge/coût, gouvernance), démontre qu'un système agentique gouverné peut concilier qualité des réponses, maîtrise des coûts et respect de la vie privée dans un cadre institutionnel.

Abstract (English)

This paper presents **ENSAM-Agent-OS**, a local agentic operating system designed for the university context of ENSAM Meknès. Facing the limitations of monolithic LLM-based solutions, we propose a modular architecture integrating six specialized agents, a tri-layer memory system (ephemeral, semantic, knowledge graph), a Retrieval-Augmented Generation (RAG) pipeline, a Model Context Protocol (MCP) tool registry, and an inter-agent debate mechanism.

The core innovation lies in **system governance** : hybrid local/cloud routing with PII detection, complete decision traceability, real-time cost tracking, GDPR compliance with right-to-be-forgotten, and a real-time observability dashboard.

Our prototype, evaluated across five real-world usage scenarios, demonstrates that a governed agentic system can reconcile response quality, cost control, and privacy compliance within an institutional framework.

Mots-clés : IA agentique, LLM, mémoire, RAG, MCP, A2A, gouvernance, observabilité, GDPR, système multi-agents, université.

Table des matières

Résumé	1
Glossaire et Acronymes	5
1 Introduction	6
1.1 Contexte général	6
1.2 Limites des approches existantes	6
1.2.1 Absence de gouvernance	6
1.2.2 Confidentialité insuffisante	7
1.2.3 Intégration fragmentée	7
1.2.4 Coûts incontrôlés	7
1.2.5 Absence de mémoire institutionnelle	7
1.3 Problématique de recherche	7
1.4 Hypothèses de recherche	8
1.5 Contributions	8
1.6 Organisation du rapport	9
2 État de l’art	9
2.1 Systèmes multi-agents à base de LLM	9
2.1.1 Frameworks multi-agents existants	10
2.1.2 Paradigmes de raisonnement agentique	11
2.2 Génération augmentée par récupération (RAG)	11
2.2.1 Principe fondamental	11
2.2.2 Évolution des architectures RAG	11
2.3 Mémoire dans les systèmes agentiques	12
2.3.1 Nécessité de la mémoire	12
2.3.2 Taxonomie des mémoires agentiques	12
2.3.3 MemGPT et l’analogie système d’exploitation	12
2.3.4 Generative Agents	13
2.4 Protocoles de communication	13
2.4.1 Model Context Protocol (MCP)	13
2.4.2 Agent-to-Agent Protocol (A2A)	13
2.5 Observabilité des systèmes LLM	14
3 Méthodologie	14
3.1 Approche de conception	14
3.2 Architecture proposée — Vue d’ensemble	14
3.3 Choix technologiques	15

3.4	Protocole expérimental	16
3.4.1	Scénarios d'évaluation	16
3.4.2	Métriques de performance	17
4	Implémentation	17
4.1	Le kernel agentique (AgenticKernel)	18
4.1.1	Bus d'événements (EventBus)	19
4.2	Système d'agents spécialisés	20
4.2.1	Architecture BaseAgent	20
4.2.2	Classification d'intention	21
4.2.3	Agents spécialisés	23
4.2.4	Le DebateEngine : communication inter-agents	23
4.3	Mémoire tri-couche	24
4.3.1	Flux de données mémoire	24
4.4	Pipeline RAG	25
4.4.1	Ingestion automatique	25
4.5	Registre d'outils MCP	26
4.5.1	Architecture du registre	26
4.5.2	Serveurs MCP implémentés	27
4.6	Routage LLM hybride	27
4.6.1	Estimation de complexité	28
4.6.2	Fallback automatique	28
4.7	Observabilité et gouvernance	29
4.7.1	Format d'une entrée d'audit	29
4.8	Conformité GDPR et droit à l'oubli	30
4.9	Interface utilisateur — Tableau de bord	30
5	Expérimentation	31
5.1	Environnement expérimental	31
5.2	Jeux de données documentaires	32
5.3	Protocole de test détaillé	32
5.3.1	S1 — Tutorat : qualité pédagogique	32
5.3.2	S2 — Recherche documentaire	33
5.3.3	S3 — Administration et outils MCP	33
5.3.4	S4 — Charge	33
5.3.5	S5 — Gouvernance et conformité GDPR	34
6	Résultats et discussion	34
6.1	Classification d'intention	34
6.2	Performance du routage hybride	35

6.3	Évaluation du RAG	36
6.4	Analyse des compromis	36
6.4.1	Synthèse des compromis systémiques	36
6.5	Évaluation de la gouvernance (S5)	37
6.6	Limites identifiées	38
7	Conclusion et perspectives	39
7.1	Synthèse des contributions	39
7.2	Validation des hypothèses	39
7.3	Réponse aux questions de recherche	40
7.4	Perspectives	40
7.4.1	Court terme (1–3 mois)	40
7.4.2	Moyen terme (3–6 mois)	41
7.4.3	Long terme (6–12 mois)	41
A	Structure complète du code source	43
B	Endpoints API complets	45
C	Diagramme de séquence — Traitement d’une requête	46

Glossaire et Acronymes

Terme	Définition
Agent	Entité logicielle autonome capable de percevoir son environnement, de raisonner et d'agir pour atteindre un objectif défini.
LLM	<i>Large Language Model</i> — Grand modèle de langage pré-entraîné sur de vastes corpus textuels (GPT, Llama, Qwen, etc.).
RAG	<i>Retrieval-Augmented Generation</i> — Technique combinant la récupération d'information dans une base documentaire avec la génération par un LLM.
MCP	<i>Model Context Protocol</i> — Protocole standardisé (Anthropic, 2024) permettant aux LLM d'accéder à des outils et sources de données.
A2A	<i>Agent-to-Agent Protocol</i> — Protocole standardisé (Google, 2025) pour la communication entre agents autonomes.
Embedding	Représentation vectorielle dense d'un texte dans un espace continu, permettant le calcul de similarité sémantique.
Chunking	Découpage d'un document en fragments de taille contrôlée pour l'indexation vectorielle.
PII	<i>Personally Identifiable Information</i> — Données permettant d'identifier une personne (nom, numéro étudiant, email, etc.).
GDPR / RGPD	<i>General Data Protection Regulation</i> — Règlement européen sur la protection des données personnelles.
Kernel	Noyau central du système agentique, responsable de l'orchestration de tous les sous-systèmes.
Ollama	Moteur d'inférence LLM local permettant d'exécuter des modèles sur un PC sans connexion cloud.
Groq	Service cloud d'inférence LLM à très faible latence basé sur des puces LPU spécialisées.
ChromaDB	Base de données vectorielle embarquée, conçue pour le stockage et la recherche d'embeddings.
NetworkX	Bibliothèque Python pour la création et la manipulation de graphes complexes.

Terme	Définition
FastAPI	Framework web Python haute performance, basé sur les standards OpenAPI et les types Python.
OpenTelemetry	Standard ouvert d'instrumentation pour la collecte de traces, métriques et logs.

1 Introduction

1.1 Contexte général

L'essor des grands modèles de langage (LLM) a profondément transformé le paysage de l'intelligence artificielle appliquée [1]. Des systèmes comme ChatGPT (OpenAI), Claude (Anthropic), Gemini (Google) ou Llama (Meta) [2] sont désormais capables de comprendre et de générer du texte naturel avec une fluidité remarquable, ouvrant des perspectives considérables pour l'automatisation de tâches intellectuelles complexes.

Dans le domaine de l'enseignement supérieur, ces modèles offrent des applications prometteuses : aide à la compréhension de cours, génération d'exercices personnalisés, synthèse de documents scientifiques, assistance à la rédaction, tutorat adaptatif et orientation académique [1]. Des établissements du monde entier expérimentent l'intégration de ces outils dans leurs parcours pédagogiques.

Cependant, l'utilisation institutionnelle de ces technologies se heurte à des obstacles fondamentaux que les solutions commerciales ne résolvent pas. La question n'est plus de savoir *si* les LLM sont utiles dans l'éducation, mais *comment* les déployer de manière responsable, contrôlée et conforme aux exigences d'un établissement public.

1.2 Limites des approches existantes

Les solutions actuellement disponibles présentent cinq limites majeures pour un déploiement universitaire :

1.2.1 Absence de gouvernance

Les systèmes commerciaux (ChatGPT, Copilot, Gemini) fonctionnent comme des boîtes noires. Il est impossible de savoir :

- Quelles sources ont été utilisées pour générer une réponse (problème de *provenance*).
- Quel raisonnement a conduit à une décision de routage ou de réponse (problème de *traçabilité*).

- Combien coûte réellement chaque interaction (problème de *maîtrise budgétaire*).
- Qui a eu accès aux données échangées (problème de *responsabilité*).

1.2.2 Confidentialité insuffisante

Les données étudiantes — notes, parcours académiques, questions personnelles, identifiants institutionnels — transitent par des serveurs tiers sans garantie de conformité avec le Règlement Général sur la Protection des Données (GDPR). Pour un établissement public marocain, cette absence de contrôle sur les données constitue un risque juridique et éthique majeur.

1.2.3 Intégration fragmentée

Les frameworks existants excellent dans un domaine spécifique mais ne proposent pas une pile intégrée. LangChain [3] facilite le chaînage de composants, AutoGen [4] orchestre des conversations multi-agents, CrewAI [6] simplifie la définition de rôles. Aucun n'intègre nativement mémoire persistante + RAG + MCP + observabilité + conformité GDPR dans un système cohérent.

1.2.4 Coûts incontrôlés

L'utilisation intensive d'API cloud (\$0.01–\$0.06 par requête pour GPT-4) peut engendrer des factures mensuelles imprévisibles, incompatibles avec les budgets d'un établissement public. Sans mécanisme de routage intelligent, chaque requête, même triviale, consomme des ressources cloud coûteuses.

1.2.5 Absence de mémoire institutionnelle

Les chatbots commerciaux n'ont pas de mémoire persistante au-delà d'une conversation. Chaque nouvelle session repart de zéro, sans contexte sur l'étudiant, ses cours précédents, ses difficultés identifiées ou son parcours. Cette amnésie empêche toute personnalisation à long terme.

1.3 Problématique de recherche

Ces constats convergent vers une problématique centrale :

Problématique

Comment construire une plateforme agentique complète, cohérente et gouvernée, capable de gérer mémoire, outils (tools), RAG, MCP et communication inter-agents (A2A) dans un cadre institutionnel universitaire, tout en garantissant traçabilité, maîtrise des coûts et conformité GDPR ?

Cette problématique ne relève pas de la simple juxtaposition de composants technologiques, mais bien d'un problème d'**intégration système** : comment faire coexister ces briques de manière harmonieuse, avec des politiques de gouvernance transversales.

1.4 Hypothèses de recherche

Nous formulons une hypothèse principale décomposée en quatre sous-hypothèses testables :

Hypothèse principale

Un système agentique modulaire, organisé comme un système d'exploitation (kernel, registre de services, politique mémoire, observabilité), permet de concilier qualité des réponses, traçabilité des décisions, contrôle des coûts et conformité GDPR dans un contexte universitaire.

- H1 — Modularité** Une architecture en couches (kernel → agents → services) permet d'ajouter ou de remplacer des composants sans affecter le reste du système.
- H2 — Qualité** L'intégration mémoire + RAG + outils améliore la pertinence des réponses par rapport à un LLM nu (sans contexte).
- H3 — Coût** Le routage hybride local/cloud réduit les coûts d'au moins 50% sans dégradation significative de la qualité perçue.
- H4 — Gouvernance** La traçabilité, l'audit et la conformité GDPR sont réalisables avec un overhead de latence inférieur à 10%.

1.5 Contributions

Ce travail apporte les contributions suivantes :

1. **Architecture AgenticKernel** : un noyau central orchestrant agents, mémoire, RAG, outils et observabilité selon le modèle d'un système d'exploitation — inspiré des travaux de Packer et al. sur MemGPT [8].
2. **Routage LLM hybride PII-aware** : un mécanisme de routage intelligent entre inférence locale (Ollama) et cloud (Groq), avec détection automatique de données

personnelles via expressions régulières adaptées au contexte marocain (CNE, CIN, etc.).

3. **Mémoire tri-couche** : intégration de trois niveaux de mémoire (éphémère avec TTL, sémantique via ChromaDB, graphe de connaissances via NetworkX) avec une façade unifiée et un mécanisme GDPR transversal.
4. **Gouvernance intégrée** : traçabilité complète des décisions (audit), suivi des coûts par requête, métriques de performance en temps réel, et droit à l'oubli conforme à l'article 17 du RGPD — le tout exposé via un tableau de bord interactif.
5. **Prototype fonctionnel** : un système complet (backend FastAPI + frontend Next.js) déployable localement, évalué sur cinq scénarios d'usage réels issus du contexte ENSAM Meknès.

1.6 Organisation du rapport

Le présent rapport est organisé en sept chapitres. Le chapitre 2 dresse un état de l'art des systèmes agentiques, de la mémoire, du RAG, des protocoles MCP/A2A et de l'observabilité. Le chapitre 3 détaille notre méthodologie et notre protocole expérimental. Le chapitre 4 décrit l'implémentation détaillée de chaque composant du système. Le chapitre 5 présente les scénarios d'évaluation et les jeux de données. Le chapitre 6 analyse les résultats obtenus et les compromis observés. Enfin, le chapitre 7 synthétise nos apports et ouvre des perspectives de recherche et de développement.

2 État de l'art

Ce chapitre présente une synthèse des concepts, travaux et technologies qui constituent le socle de notre système. Nous couvrons successivement les systèmes multi-agents, le RAG, la mémoire, les protocoles MCP et A2A, l'observabilité, et le positionnement de notre contribution.

2.1 Systèmes multi-agents à base de LLM

L'IA agentique représente un changement de paradigme où les LLM passent du rôle de générateurs de texte passifs à celui d'*agents autonomes* capables de percevoir, raisonner, planifier et agir [9, 10]. Un agent, au sens de l'IA, est une entité qui :

1. **Perçoit** son environnement (requêtes utilisateur, contexte, données).
2. **Raisonne** sur la meilleure action à entreprendre (Chain-of-Thought [11], ReAct [12]).
3. **Agit** en utilisant des outils, en interrogeant des bases de données ou en communiquant avec d'autres agents.

4. **Mémorise** les interactions pour améliorer ses réponses futures.

2.1.1 Frameworks multi-agents existants

Plusieurs frameworks ont émergé pour orchestrer des systèmes multi-agents :

AutoGen [4] (Microsoft Research, 2023) propose des conversations multi-agents avec des patterns programmables (séquentiel, groupe, hiérarchique). Sa force réside dans la flexibilité des topologies de conversation et le support de l'intervention humaine (*human-in-the-loop*). Cependant, AutoGen ne fournit ni mémoire persistante native, ni gouvernance des coûts, ni conformité GDPR.

MetaGPT [5] (2023) modélise les agents comme des rôles professionnels (architecte, développeur, testeur) avec des SOP (*Standard Operating Procedures*) qui structurent leur collaboration. L'approche est puissante pour le développement logiciel mais peu adaptable à d'autres domaines comme l'éducation.

CrewAI [6] (2024) simplifie la création d'équipes d'agents avec des rôles, objectifs et outils définis déclarativement. Son API intuitive a favorisé une adoption rapide. Cependant, CrewAI repose entièrement sur des API cloud sans option de routage local.

LangGraph [7] (LangChain, 2024) modélise les workflows agentiques comme des graphes d'états avec persistance, permettant des flux complexes avec boucles, points de contrôle et intervention humaine. C'est le framework le plus proche de notre approche architecturale, mais il ne fournit pas de gouvernance intégrée, de suivi des coûts natif, ni de conformité GDPR.

Table 2 – Comparaison détaillée des frameworks multi-agents existants

Critère	<i>AutoGen</i>	<i>MetaGPT</i>	<i>CrewAI</i>	<i>LangGraph</i>	<i>LangChain</i>	<i>ENSAM-OS</i>
Multi-agents	✓	✓	✓	✓	~	✓
Mémoire persistante	×	×	~	✓	~	✓
RAG intégré	~	×	✓	✓	✓	✓
Tools / MCP	✓	✓	✓	✓	✓	✓
Observabilité	×	×	~	~	~	✓
Suivi des coûts	×	×	×	×	~	✓
Routage local/cloud	×	×	×	×	×	✓
GDPR natif	×	×	×	×	×	✓
Tableau de bord	×	×	×	~	~	✓

✓ = support natif complet ~ = partiel ou via plugin × = absent

2.1.2 Paradigmes de raisonnement agentique

Les agents s'appuient sur des paradigmes de raisonnement pour structurer leurs actions :

Chain-of-Thought (CoT) [11] incite le modèle à décomposer son raisonnement en étapes explicites avant de produire une réponse finale. Cette technique améliore significativement les performances sur les tâches de raisonnement mathématique et logique.

ReAct [12] (*Reasoning + Acting*) alterne entre des phases de raisonnement textuel et des actions concrètes (appels d'outils, recherches). Ce paradigme est fondamental pour les agents qui interagissent avec des environnements externes.

Toolformer [13] démontre que les LLM peuvent apprendre de manière auto-supervisée quand et comment utiliser des outils (calculatrice, API de recherche, traducteur). Cette capacité est au cœur de l'intégration d'outils dans les agents.

2.2 Génération augmentée par récupération (RAG)

2.2.1 Principe fondamental

Le RAG [14] combine deux mécanismes complémentaires : la **récupération** (*retrieval*) d'informations pertinentes dans une base documentaire, et la **génération** d'une réponse par un LLM en s'appuyant sur les documents récupérés. Cette approche résout deux problèmes majeurs des LLM :

- Les **hallucinations** : en fondant la réponse sur des sources vérifiables.
- La **péremption** des connaissances : en permettant l'accès à des documents mis à jour sans ré-entraînement du modèle.

2.2.2 Évolution des architectures RAG

Gao et al. [15] et Fan et al. [16] distinguent trois générations d'architectures RAG :

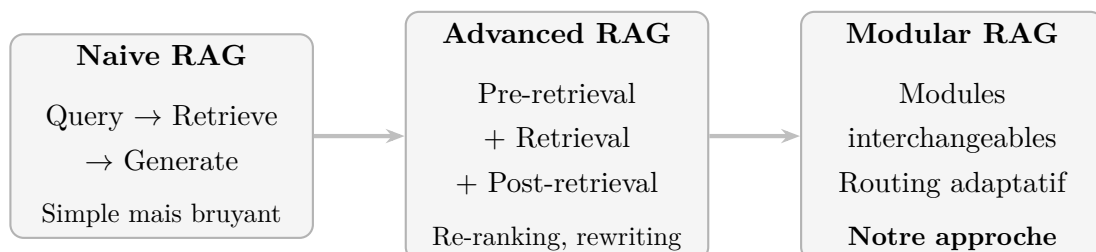


Figure 1 – Évolution des architectures RAG [15]

Notre implémentation s'inscrit dans la catégorie **Modular RAG** : le pipeline est composé de modules indépendants (parseur, chunker, embedder, retriever, générateur) qui peuvent

être remplacés ou configurés indépendamment.

2.3 Mémoire dans les systèmes agentiques

2.3.1 Nécessité de la mémoire

Sans mémoire, un agent ne peut ni apprendre de ses interactions passées, ni personnaliser ses réponses, ni maintenir la cohérence d'une conversation longue. La mémoire est ce qui transforme un chatbot sans état en un assistant véritablement intelligent.

2.3.2 Taxonomie des mémoires agentiques

Zhang et al. [17] catégorisent les mémoires agentiques selon trois dimensions :

Table 3 – *Taxonomie des mémoires agentiques [17]*

Type	Durée	Mécanisme	Analogie humaine
Court terme	Session	Buffer de conversation, fenêtre de contexte	Mémoire de travail
Long terme — sémantique	Persistente	Base vectorielle, embeddings	Mémoire déclarative
Long terme — épisodique	Persistente	Événements datés, séquences	Mémoire épisodique
Long terme — procédurale	Persistente	Règles, politiques, habitudes	Mémoire procédurale
Graphe	Persistente	Entités, relations, ontologies	Mémoire sémantique

2.3.3 MemGPT et l'analogie système d'exploitation

Packer et al. [8] proposent avec MemGPT une analogie fondamentale avec la gestion mémoire des systèmes d'exploitation : la fenêtre de contexte du LLM est assimilée à la mémoire principale (RAM) — limitée mais rapide — tandis qu'un stockage externe joue le rôle de mémoire secondaire (disque) — illimitée mais plus lente. Un mécanisme de « pagination » gère les transferts entre les deux niveaux.

Cette analogie inspire directement notre architecture, que nous étendons avec un troisième niveau : le **graphe de connaissances**, qui capture les relations structurées entre entités (étudiants, cours, documents, agents) et enrichit le contexte au-delà de la simple similarité textuelle.

2.3.4 Generative Agents

Park et al. [18] ont démontré avec les *Generative Agents* que la combinaison mémoire épisodique + mécanisme de réflexion produit des comportements émergents réalistes. Leur architecture comprend un flux de mémoire (observations horodatées), un mécanisme de récupération (par pertinence, récence et importance), et un module de réflexion (synthèse de haut niveau). Notre mémoire sémantique s'inspire de leur mécanisme de récupération par pertinence.

2.4 Protocoles de communication

2.4.1 Model Context Protocol (MCP)

Le **Model Context Protocol** [19], proposé par Anthropic en novembre 2024, standardise la manière dont les LLM accèdent à des outils et des sources de données contextuelles. Le protocole définit :

- **Tools** : fonctions invocables par le modèle (ex : `schedule_lookup`, `web_search`).
- **Resources** : sources de données lisibles (fichiers, bases de données).
- **Prompts** : templates de prompts pré-configurés.
- **Transport** : communication via stdio ou HTTP/SSE.

L'adoption du MCP s'accélère : les principaux IDE (VS Code, Cursor, Windsurf) et frameworks (LangChain, CrewAI) l'intègrent progressivement. Notre système implémente un sous-ensemble du protocole axé sur la découverte et l'exécution d'outils avec des niveaux de sécurité.

2.4.2 Agent-to-Agent Protocol (A2A)

Le protocole **Agent-to-Agent** [20], proposé par Google en avril 2025, standardise la communication entre agents autonomes via :

- **Agent Cards** : documents JSON décrivant les capacités, compétences et endpoints d'un agent.
- **Task Protocol** : mécanisme de délégation de tâches entre agents avec suivi d'état (submitted, working, completed, failed).
- **Messages** : échanges structurés avec pièces jointes typées.

Dans notre système, la communication inter-agents s'effectue principalement via deux mécanismes : (1) le **routing dynamique** du kernel, qui délègue les requêtes à l'agent spécialisé approprié en fonction de l'intention détectée, et (2) le **DebateEngine**, qui orchestre

des confrontations argumentées multi-rounds entre perspectives opposées. Ces mécanismes constituent une forme de communication A2A interne, bien qu'ils n'implémentent pas le protocole formel de Google.

2.5 Observabilité des systèmes LLM

L'observabilité — capacité à comprendre l'état interne d'un système à partir de ses sorties — est un domaine émergent pour les systèmes à base de LLM.

LangSmith [21] (LangChain) propose du tracing de chaînes, de l'évaluation automatique et du monitoring. Cependant, il est lié à l'écosystème LangChain et ne fournit pas de suivi des coûts par requête ni de conformité GDPR.

OpenTelemetry [22] fournit un standard industriel pour l'instrumentation (traces, métriques, logs) mais n'est pas spécialisé pour les LLM. L'adaptation à un système agentique nécessite un travail d'intégration significatif.

Positionnement de notre contribution

Aucune solution existante n'intègre nativement dans un même système :

- Suivi des coûts **par requête** avec calcul d'économies
- Routage **PII-aware** entre local et cloud
- Audit trail avec **provenance des réponses**
- Conformité GDPR avec **droit à l'oubli**
- Tableau de bord d'observabilité **temps réel**

C'est précisément ce créneau que notre système occupe.

3 Méthodologie

3.1 Approche de conception

Notre approche de conception suit le modèle itératif *Design Science Research* : (1) identification du problème, (2) définition des objectifs, (3) conception de l'artefact, (4) développement, (5) évaluation, (6) communication. Le système a été développé en sprints de deux semaines, chaque sprint ajoutant une brique de la pile complète.

3.2 Architecture proposée — Vue d'ensemble

L'architecture d'ENSAM-Agent-OS s'organise en cinq couches horizontales, traversées par un axe d'observabilité vertical :

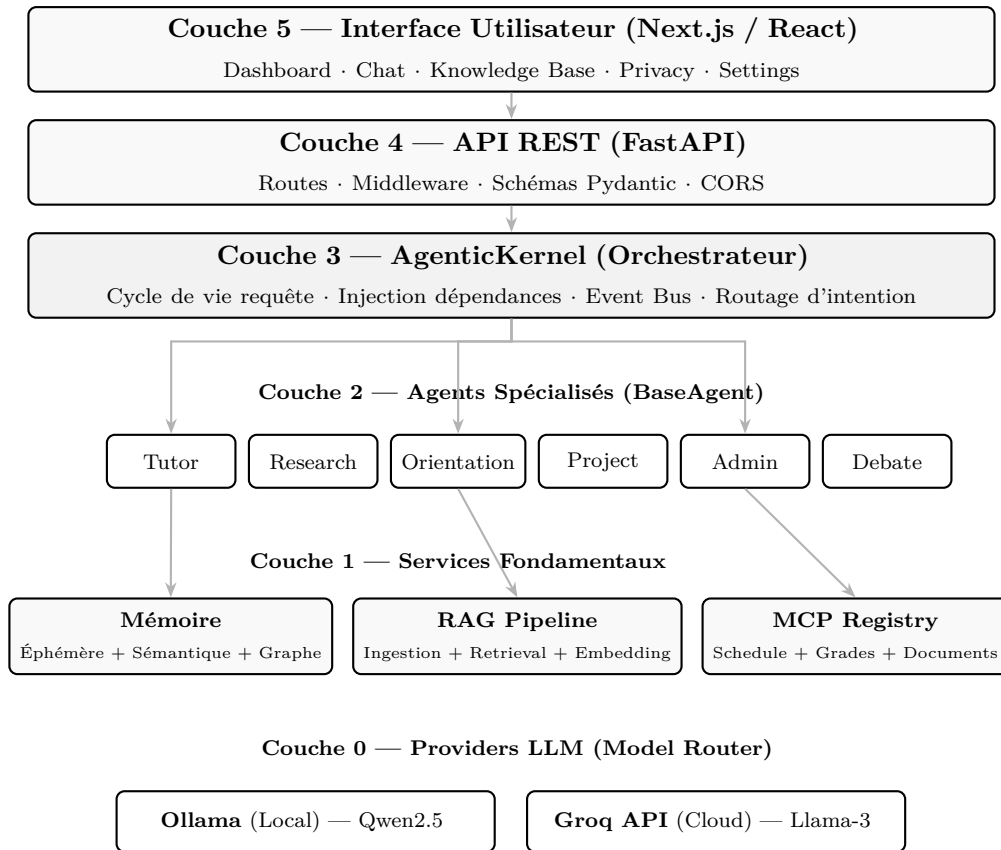


Figure 2 – Architecture en couches d’ENSAM-Agent-OS. L’observabilité (audit, métriques, coûts et conformité GDPR) constitue une préoccupation transversale à l’ensemble des couches.

3.3 Choix technologiques

Chaque brique technologique a été sélectionnée selon trois critères : (1) adéquation fonctionnelle, (2) légèreté et déployabilité locale, (3) maturité et communauté active.

Table 4 – *Pile technologique et justifications détaillées*

Couche	Composant	Technologie	Justification
Backend	Framework web	FastAPI	Async natif, validation Pydantic, OpenAPI auto, performances élevées
Backend	LLM local	Ollama + Qwen2.5	Inférence locale, 0 coût, confidentialité totale, support multilingue FR/AR
Backend	LLM cloud	Groq API	Latence <500ms grâce aux puces LPU, modèles Llama-3/Mixtral
Backend	Base vectorielle	ChromaDB	Embarquée, sans serveur, HNSW pour recherche rapide
Backend	Graphe	NetworkX	Pur Python, sérialisation JSON, sans dépendance
Backend	Classification	Sentence-BERT	Multilingue, rapide (~5ms CPU), pas d'appel LLM pour le routage
Backend	Chunking	LangChain Splitter	RecursiveCharacterTextSplitter robuste et paramétrable
Backend	Observabilité	OpenTelemetry	Standard industriel CNCF, extensible, vendor-neutral
Frontend	Framework	Next.js 16	SSR/CSR, App Router, écosystème riche
Frontend	UI	React 19 + Tailwind	Composants réactifs, styling utilitaire
Frontend	Graphiques	Recharts	Graphiques déclaratifs pour le dashboard
Frontend	Graphe	ReactFlow	Visualisation interactive du graphe de connaissances
Frontend	État	TanStack Query	Gestion du cache serveur, refetching automatique

3.4 Protocole expérimental

3.4.1 Scénarios d'évaluation

L'évaluation s'effectue selon cinq scénarios couvrant les cas d'utilisation principaux du contexte universitaire :

Table 5 – Scénarios d’évaluation et métriques associées

#	Scénario	Objectif	Métriques clés
S1	Tutorat	Qualité pédagogique des réponses et pertinence RAG	Precision@k, score RAG
S2	Recherche	Recherche documentaire multi-sources	Recall, diversité sources
S3	Administration	Exactitude des outils MCP (notes, EDT)	Exactitude, latence outil
S4	Charge/Coût	Distribution routage et coût sous charge	Coût/req, latence p95
S5	Gouvernance	Conformité GDPR, audit trail	Complétude effacement

3.4.2 Métriques de performance

- **Latence** : temps total de traitement d’une requête (p50, p95, p99).
- **Coût** : coût en dollars par requête, calculé selon le provider et les tokens consommés.
- **Tokens** : nombre de tokens d’entrée et de sortie par requête.
- **Taux de cache** : pourcentage de requêtes servies depuis un cache (mémoire ou embedding).
- **Précision de classification** : pourcentage de requêtes routées vers le bon agent.
- **Score de pertinence RAG** : score cosinus moyen des chunks récupérés (0 à 1).
- **Overhead de gouvernance** : latence additionnelle due à l’audit, au tracking et à la journalisation.

4 Implémentation

Ce chapitre détaille l’implémentation de chaque composant du système. La structure du code source suit l’organisation suivante :

Listing 1 – Structure du dépôt de code

```

1 ENSAM-Agent-OS/
2   backend/
3     app/
4       core/           # Kernel, routing, LLM, events
5       agents/         # Agents specialises + classifieur
6       memory/         # 3 couches + GDPR
7       rag/            # Pipeline RAG complet
8       mcp/            # Registry + serveurs MCP
9       observability/  # Audit, couts, metriques, telemetry
10      api/             # Routes FastAPI + middleware
11      data/            # Documents pour le RAG
12      pyproject.toml

```

```

13 frontend/
14     src/app/          # Pages Next.js
15     src/components/   # Composants React
16     src/lib/          # Client API + providers
17 article/             # Ce rapport
18 run.bat              # Script de démarrage

```

4.1 Le kernel agentique (AgenticKernel)

Le cœur du système est la classe `AgentKernel`, inspirée de l'analogie avec un noyau de système d'exploitation [8]. Le kernel est responsable de :

1. **Initialisation** : instantiation et câblage de tous les sous-systèmes au démarrage.
2. **Orchestration** : coordination du pipeline de traitement de chaque requête.
3. **Cycle de vie** : gestion du démarrage (**startup**) et de l'arrêt (**shutdown**) propres.

Listing 2 – Pipeline de traitement du kernel (simplifié)

```

1 class AgentKernel:
2     """Noyau central du systeme agentique."""
3
4     def __init__(self):
5         self.llm_manager: LLMManager = None
6         self.router: ModelRouter = None
7         self.memory: MemoryManager = None
8         self.rag: RAGPipeline = None
9         self.mcp: MCPRegistry = None
10        self.cost_tracker: CostTracker = None
11        self.audit_logger: AuditLogger = None
12        self.event_bus: EventBus = None
13
14    async def process_request(
15        self, message: str, session_id: str, user_id: str
16    ) -> dict:
17        context = RequestContext(
18            session_id=session_id,
19            user_id=user_id,
20            request_id=str(uuid.uuid4())
21        )
22        try:
23            # 1. Charger l'historique de session
24            history = await self.memory.get_history(session_id)
25
26            # 2. Determiner le routage LLM (local vs cloud)

```

```
27         routing = self.router.analyze(message)
28
29         # 3. Generer la reponse via le LLM selectionne
30         response = await self.llm_manager.generate(
31             prompt=message,
32             history=history,
33             target=routing.target
34         )
35
36         # 4. Sauvegarder dans la memoire
37         await self.memory.save(session_id, message, response)
38
39         # 5. Enregistrer le cout
40         self.cost_tracker.record(routing, response)
41
42         # 6. Logger dans l'audit trail
43         self.audit_logger.log_request(context, routing, response)
44
45         # 7. Publier l'evenement
46         await self.event_bus.publish("response_generated", {...})
47
48         return {"response": response.text, "trace": context}
49
50     except Exception as e:
51         self.audit_logger.log_error(context, e)
52         raise
```

4.1.1 Bus d'événements (EventBus)

Le kernel utilise un bus d'événements asynchrone pour découpler les composants. Douze types d'événements sont définis :

Table 6 – Types d'événements du système

Événement	Description	Émetteur
message_received	Nouvelle requête utilisateur	API
agent_selected	Agent choisi par le classifieur	Kernel
response_generated	Réponse LLM produite	Agent
memory_stored	Données sauvegardées en mémoire	MemoryManager
rag_retrieved	Documents RAG récupérés	RAGPipeline
tool_called	Outil MCP invoqué	MCPClient
cost_recorded	Coût enregistré	CostTracker
error_occurred	Erreur survenue	Tout composant
cost_alert	Seuil de budget dépassé	CostTracker
pii_detected	Données personnelles détectées	ModelRouter
gdpr_erasure	Effacement GDPR exécuté	GDPRManager
agent_registered	Nouvel agent enregistré	EventBus

4.2 Système d'agents spécialisés

4.2.1 Architecture BaseAgent

Tous les agents héritent d'une classe abstraite **BaseAgent** qui implémente le patron *Template Method*. Ce patron définit le squelette algorithmique du traitement d'une requête tout en déléguant les spécificités (prompt système, sélection d'outils, politique RAG) aux sous-classes.

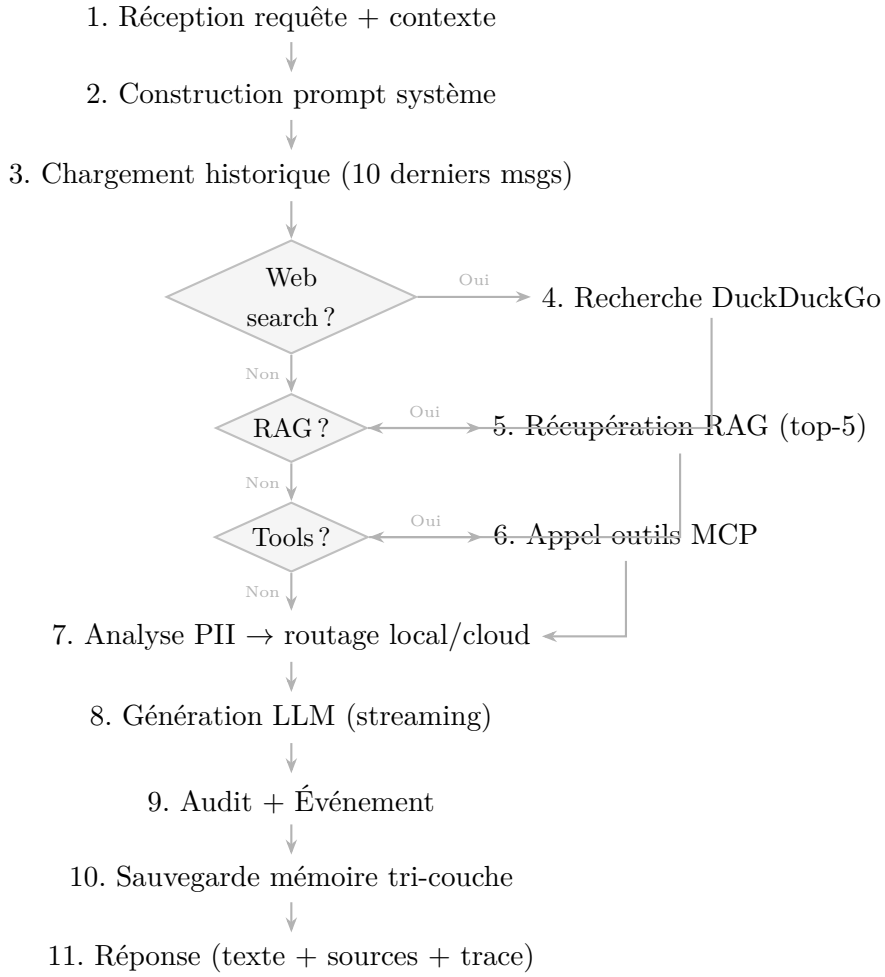


Figure 3 – Pipeline complet de traitement d’une requête par un agent

4.2.2 Classification d’intention

La classification utilise un modèle d’embeddings multilingue `paraphrase-multilingual-MiniLM-L12-v2` chargé localement. Le processus se déroule en quatre étapes :

1. **Enregistrement** : chaque agent fournit 6 à 10 phrases exemplaires bilingues (FR/EN) lors de l’initialisation.
2. **Pré-calcul** : les embeddings de tous les exemplaires sont calculés et stockés en mémoire (matrice $N_{agents} \times N_{exemplars} \times 384$).
3. **Classification** : pour chaque requête, son embedding est comparé à tous les exemplaires par similarité cosinus :

$$\text{sim}(q, e) = \frac{q \cdot e}{\|q\| \cdot \|e\|} \quad (1)$$

4. **Décision** : l’agent ayant le score maximal est sélectionné si $\max(\text{sim}) > 0.35$; sinon, un fallback par mots-clés est tenté.

Listing 3 – Classifieur d'intention par embeddings

```

1 class IntentClassifier:
2     """Classification par embeddings multilingues."""
3
4     def __init__(self):
5         self.model = None # Lazy loading
6         self.agent_embeddings: dict[str, np.ndarray] = {}
7
8     def _load_model(self):
9         """Chargement paresseux du modele."""
10        from transformers import AutoTokenizer, AutoModel
11        name = "sentence-transformers/paraphrase-multilingual-MiniLM-L12
            -v2"
12        self.tokenizer = AutoTokenizer.from_pretrained(name)
13        self.model = AutoModel.from_pretrained(name)
14
15    def register_agent(self, agent_id: str, exemplars: list[str]):
16        """Pre-calcule les embeddings exemplaires."""
17        embeddings = [self._embed(text) for text in exemplars]
18        self.agent_embeddings[agent_id] = np.stack(embeddings)
19
20    def classify(self, query: str) -> ClassificationResult:
21        """Classifie une requete par similarite cosinus."""
22        query_emb = self._embed(query)
23        best_agent, best_score = None, -1.0
24
25        for agent_id, exemplar_embs in self.agent_embeddings.items():
26            scores = cosine_similarity(query_emb, exemplar_embs)
27            max_score = float(scores.max())
28            if max_score > best_score:
29                best_score = max_score
30                best_agent = agent_id
31
32        if best_score < 0.35:
33            # Fallback par mots-cles
34            return self._keyword_fallback(query)
35
36        return ClassificationResult(
37            agent=best_agent, confidence=best_score
38        )

```

Cette approche est exécutée en $\sim 5\text{--}15\text{ms}$ sur CPU, évitant un appel LLM coûteux ($\sim 500\text{ms}$) pour le simple routage.

4.2.3 Agents spécialisés

Table 7 – *Agents spécialisés : domaine, capacités et outils*

Agent	Domaine	Capacités	RAG	MCP Tools
TutorAgent	Pédagogie	Explications de cours, exercices, correction, exemples de code	Toujours	Documents
ResearchAgent	Recherche	Bibliographie, méthodologie, citations APA/IEEE/BibTeX	Toujours	Web, Documents
OrientationAgent	Orientation	Carrières, stages, filières, spécificités ENSAM	Si pertinent	—
ProjectAgent	Projets	Planification, diagrammes UML/Mermaid, architectures	Si pertinent	—
AdminAgent	Administration	Notes, emplois du temps, procédures, inscriptions	Conditionnelle	Schedule, Grades
DebateEngine	Débat	Argumentation multi-perspectives, synthèse contradictoire	—	—

4.2.4 Le DebateEngine : communication inter-agents

Le **DebateEngine** orchestre des débats argumentés entre deux perspectives opposées. Pour N rounds :

1. **Agent A** reçoit le sujet avec l’instruction d’argumenter **pour**.
2. **Agent B** reçoit le sujet + les arguments de A et argumente **contre**.
3. Après tous les rounds, un **juge neutre** synthétise les arguments et produit une conclusion équilibrée.

Ce mécanisme constitue une forme de protocole A2A interne : les agents communiquent via des messages structurés, chaque agent ayant accès au contexte cumulé des échanges précédents.

4.3 Mémoire tri-couche

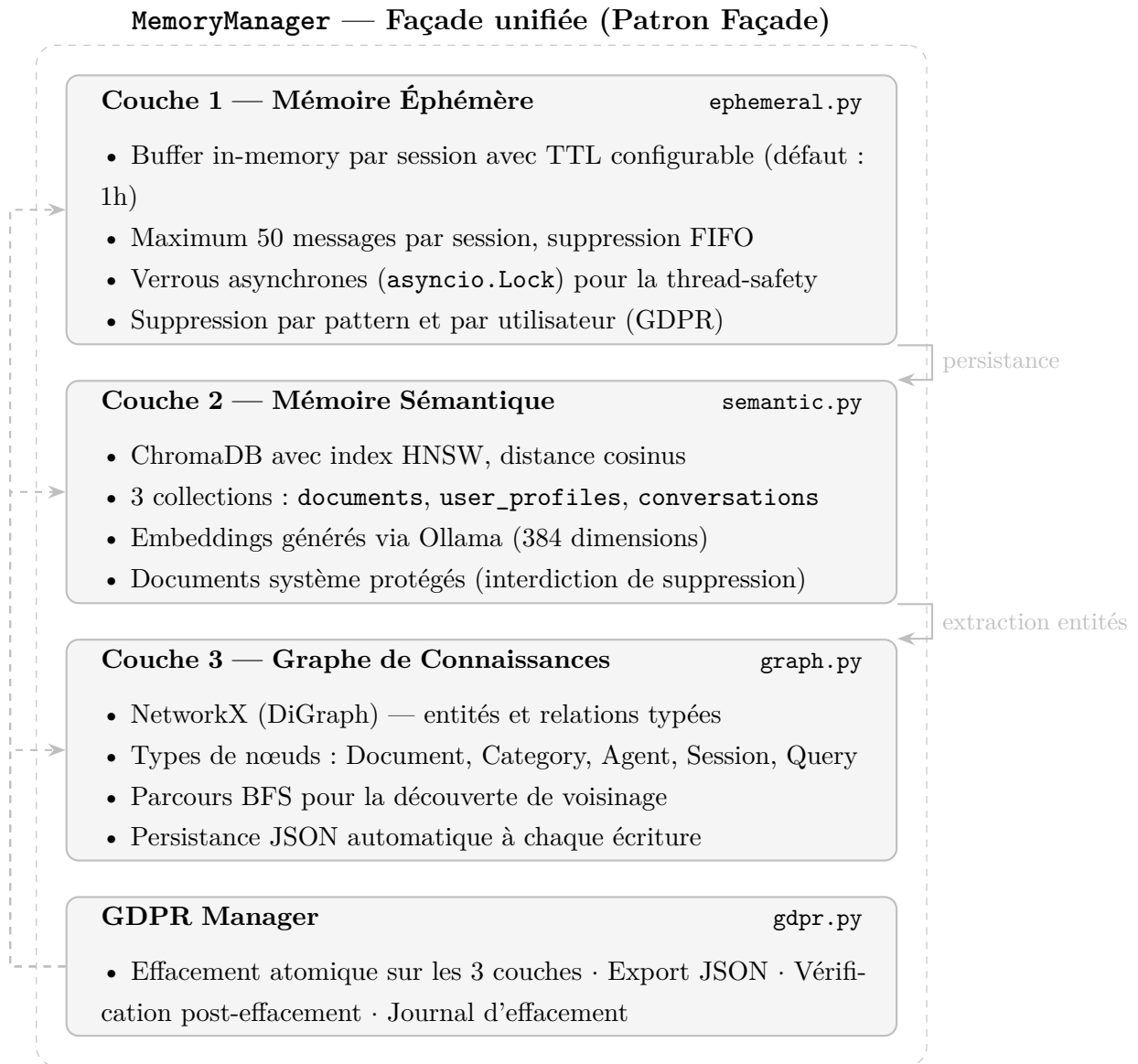


Figure 4 — Architecture détaillée de la mémoire tri-couche

4.3.1 Flux de données mémoire

Lors d'un appel `MemoryManager.store(session_id, message, response)` :

1. Le message est ajouté au **buffer éphémère** de la session (immédiat, <1ms).
2. Le message est indexé dans la **collection ChromaDB conversations** avec son embedding (~10ms).

3. Les entités extraites (noms propres, termes techniques, identifiants) sont ajoutées au **graphe NetworkX** avec des arêtes vers la session et l'agent ($\sim 5\text{ms}$).

Lors d'un appel `MemoryManager.retrieve(session_id, query)` :

1. Les derniers messages de la session sont chargés depuis le buffer éphémère.
2. Les interactions passées les plus similaires sont récupérées depuis ChromaDB.
3. Les entités et relations pertinentes sont extraites du graphe par parcours BFS.
4. Les trois sources sont fusionnées dans un contexte unifié injecté dans le prompt.

4.4 Pipeline RAG

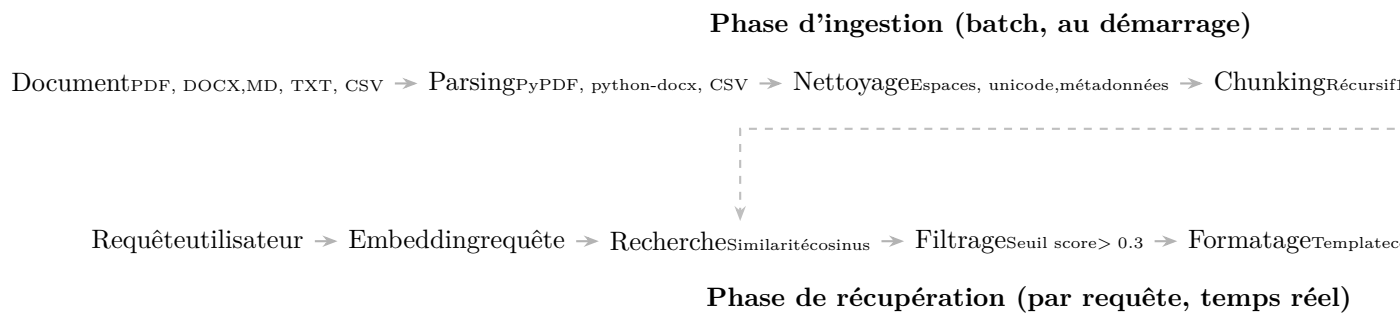


Figure 5 – *Pipeline RAG complet : ingestion et récupération*

4.4.1 Ingestion automatique

Le module `auto_ingest.py` scanne le répertoire `data/` au démarrage du serveur et indexe automatiquement les documents manquants. Les documents système (cours JAVA, UML, Time-Series, Projet-Métier, Guide ENSAM) sont marqués comme `system_document=true` et sont protégés contre la suppression par les utilisateurs.

Le chunking utilise `RecursiveCharacterTextSplitter` de LangChain avec les paramètres suivants (ajustables via le tableau de bord) :

- **Taille de chunk** : 1000 caractères (compromis entre contexte et précision).
- **Chevauchement** : 200 caractères (assure la continuité sémantique entre chunks adjacents).
- **Séparateurs** : `[\n\n, \n, ". ", " ", ""]` (découpe d'abord par paragraphe, puis par phrase).

4.5 Registre d'outils MCP

4.5.1 Architecture du registre

Le registre MCP implémente un sous-ensemble du protocole d'Anthropic [19] centré sur la découverte et l'exécution d'outils :

Listing 4 – *Définition d'un outil MCP*

```
1 @dataclass
2 class ToolDefinition:
3     id: str                # Identifiant unique
4     name: str              # Nom affichable
5     description: str       # Description pour le LLM
6     parameters: list[ToolParameter] # Schema de parametres
7     handler: Callable      # Fonction d'exécution
8     security_level: SecurityLevel # PUBLIC/STUDENT/FACULTY/ADMIN
9     tags: list[str]        # Tags de categorisation
10    is_active: bool = True  # Actif ou desactive
11
12 class SecurityLevel(str, Enum):
13     PUBLIC = "public"      # Accessible a tous
14     STUDENT = "student"    # Etudiants authentifies
15     FACULTY = "faculty"    # Enseignants uniquement
16     ADMIN = "admin"        # Administrateurs systeme
```

4.5.2 Serveurs MCP implémentés

Table 8 – *Serveurs MCP et leurs outils*

Serveur	Outil	Sécurité	Fonctionnalité	Paramètres
Schedule	schedule_lookup	STUDENT	Emploi du temps ENSAM 4A (S1/S2) par filière, jour, professeur	program, day, prof
Grades	grades_lookup	FACULTY	Consultation de notes par étudiant et matière	student_id, course
	gpa_calculator	STUDENT	Calcul GPA selon règles ENSAM (≥ 12 : valide, < 7 : éliminé)	student_id
Documents	document_search	PUBLIC	Recherche dans l'index documentaire	query, file_type
	web_search	PUBLIC	Recherche web via DuckDuckGo (scraping HTML)	query, max_results

4.6 Routage LLM hybride

Le `ModelRouter` implémente une politique de routage en cascade qui détermine si chaque requête doit être traitée localement (Ollama) ou dans le cloud (Groq).

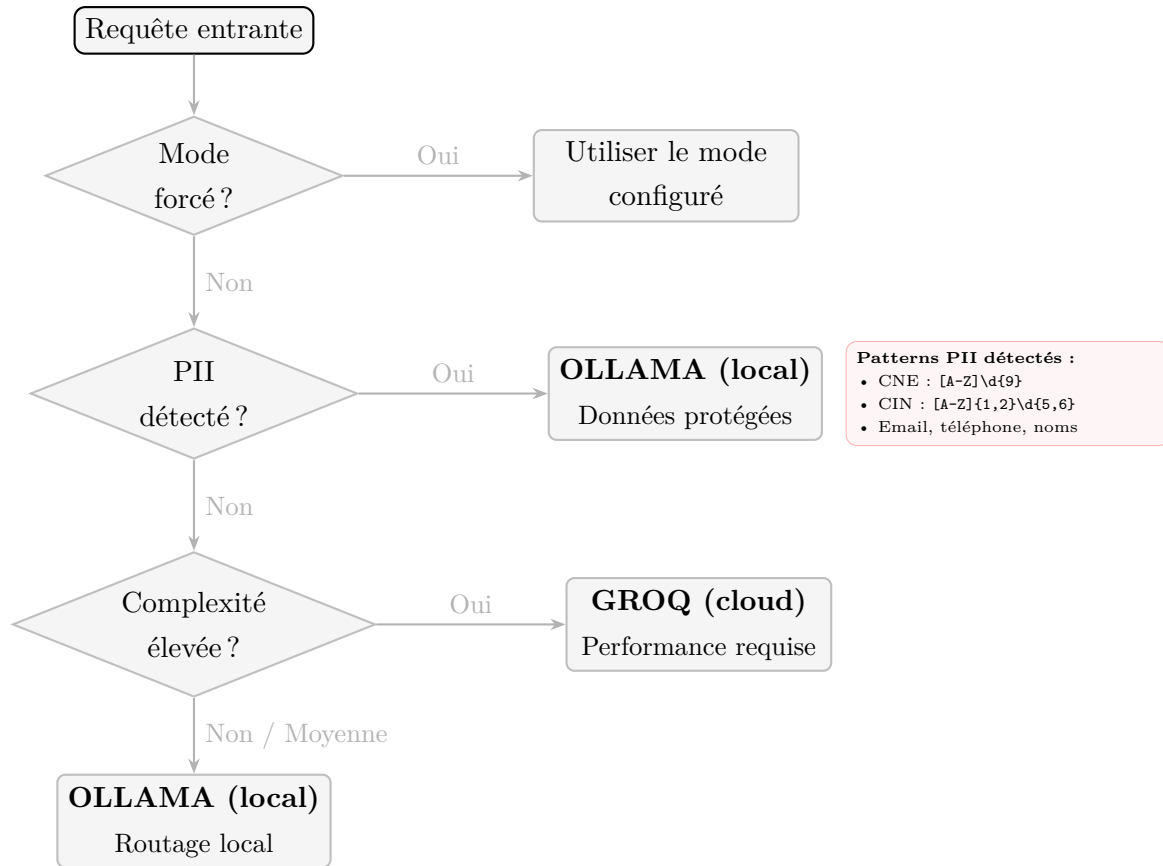


Figure 6 – Arbre de décision du routage LLM hybride PII-aware

4.6.1 Estimation de complexité

La complexité d’une requête est estimée par un score heuristique combinant :

- **Longueur** : requêtes > 200 caractères $\rightarrow +1$ point.
- **Mots-clés techniques** : présence de termes comme “analyse”, “compare”, “architecture”, “algorithmique” $\rightarrow +1$ point par mot-clé (max 3).
- **Instructions de raisonnement** : “explique en détail”, “donne un exemple”, “compare” $\rightarrow +2$ points.

Score $\geq 3 \rightarrow$ complexité haute (cloud) ; 1-2 $\rightarrow$ moyenne (cloud) ; 0 $\rightarrow$ basse (local).

4.6.2 Fallback automatique

Le **LLMManager** implémente un mécanisme de fallback : si le provider principal (déterminé par le routeur) échoue (timeout, erreur réseau, rate limit), la requête est automatiquement reroutée vers l’autre provider. Ce mécanisme garantit la disponibilité du système même en cas de panne partielle.

4.7 Observabilité et gouvernance

Le système d'observabilité constitue l'innovation principale de ce projet. Il comprend quatre composants qui fonctionnent de manière transparente, sans modifier le code métier des agents.

Module d'Observabilité

Audit Logger

- Requête → JSON structuré
- Horodatage, user, session
- Agent, routage, latence
- Tokens, coût, statut
- Export + filtrage

Cost Tracker

- Coût/requête par provider
- Groq : \$0.05/1M in, \$0.08/1M out
- Ollama : \$0.000
- Limites budget + alertes
- Rapport d'économies

Metrics Collector

- Latences : p50, p95, p99
- Fenêtre glissante (deque)
- Par provider, par agent
- CPU, RAM, disque, GPU
- Taux de cache

Telemetry (OTel)

- Spans avec contexte
- Logs structurés JSON
- Traceur configurable
- Format StructuredFormatter
- ConsoleSpanExporter

Figure 7 – Les quatre composants du système d'observabilité

4.7.1 Format d'une entrée d'audit

Chaque requête génère une entrée d'audit structurée :

Listing 5 – Exemple d'entrée d'audit JSON

```

1 {
2   "timestamp": "2025-06-18T14:32:15.123Z",
3   "request_id": "a1b2c3d4-...",
4   "user_id": "student_42",
5   "session_id": "sess_xyz",
6   "agent": "TutorAgent",
7   "action": "chat_response",
8   "routing": {
9     "target": "ollama",
10    "reason": "pii_detected",
11    "complexity": "low",
12    "pii_found": ["email"]
13  },
14  "metrics": {
15    "latency_ms": 1234,
16    "input_tokens": 256,

```

```

17     "output_tokens": 412,
18     "cost_usd": 0.000
19   },
20   "rag": {
21     "documents_retrieved": 3,
22     "avg_relevance": 0.74
23   },
24   "status": "success"
25 }

```

4.8 Conformité GDPR et droit à l’oubli

Le `GDPRManager` implémente trois fonctionnalités conformes à l’article 17 du RGPD [23] :

Table 9 – *Fonctionnalités GDPR implémentées*

Article RGPD	Fonctionnalité	Implémentation
Art. 15 — Accès	Export données	GET <code>/api/memory/gdpr/export</code> — JSON avec sessions, mémoire sémantique, nœuds graphe
Art. 17 — Effacement	Droit à l’oubli	POST <code>/api/memory/gdpr/forget</code> — Suppression atomique 3 couches avec <code>confirm=true</code> requis
Art. 17 — Vérification	Preuve d’effacement	GET <code>/api/memory/gdpr/verify</code> — Confirme l’absence de données résiduelles
—	Journal d’effacement	Chaque effacement est loggé avec horodatage, détail des données supprimées, erreurs partielles

L’effacement est conçu pour être **résilient** : si une couche échoue (ex : ChromaDB temporairement inaccessible), les couches qui ont réussi ne sont pas annulées, et les erreurs sont signalées dans le rapport d’effacement.

4.9 Interface utilisateur — Tableau de bord

Le frontend Next.js 16 comprend sept pages interconnectées. Chaque page communique avec le backend via un client API centralisé (`api.ts`) qui injecte automatiquement l’identifiant utilisateur via le header `X-User-ID`.

Table 10 – Pages du tableau de bord et fonctionnalités

Page	Fonctionnalités principales	Rafraîchissement
/ Dashboard	KPI (statut, coût total, latence moy.), navigation rapide, statut services	15s
/chat	Chat multi-sessions, sélection d’agent, sources RAG, traces, métadonnées, détection de langue FR/EN	Temps réel
/knowledge	Upload drag&drop, recherche sémantique, graphe interactif ReactFlow, config chunking	10s (graphe)
/tools	Registre MCP, test interactif, niveaux sécurité, résultats JSON	Manuel
/observability	6 KPI, graphiques Recharts (latence, providers, catégories), ressources système, événements	5s
/privacy	Périmètre données, stats mémoire, droit à l’oubli avec confirmation, audit trail	Manuel
/settings	Modèle LLM, mode routage, santé services, langue, session	Manuel

5 Expérimentation

5.1 Environnement expérimental

Table 11 – Spécifications de l’environnement de test

Composant	Spécification
Processeur	AMD Ryzen 7 / Intel Core i7 (8+ cœurs)
Mémoire vive	16 Go DDR4
GPU	NVIDIA GTX/RTX (optionnel, pour accélération Ollama)
Stockage	SSD NVMe 512 Go
Système d’exploitation	Windows 10/11
Python	3.12+ (gestionnaire uv)
Node.js	20+
Ollama	v0.5+ avec modèle Qwen2.5 :3B (2 Go VRAM)
Groq API	Llama-3.3-70B-Versatile / Mixtral-8x7B-32768
ChromaDB	v0.5+ (embarqué, mode fichier)
Navigateur	Chrome/Firefox (pour le frontend)

5.2 Jeux de données documentaires

Les documents utilisés pour le RAG proviennent du programme 4A de l'ENSAM Meknès et couvrent quatre matières :

Table 12 – *Corpus documentaire pour le RAG*

Catégorie	Docs	Chunks	Taille	Contenu
JAVA	12	~156	180 Ko	Programmation orientée objet, design patterns, collections
UML	8	~94	120 Ko	Diagrammes de classes, séquence, cas d'utilisation
Time-Series	6	~78	95 Ko	Analyse de séries temporelles, décomposition, ARIMA
Projet-Métier	5	~62	75 Ko	Gestion de projet, méthodologies agiles, livrables
Guide ENSAM	1	~35	8 Ko	Guide général de l'établissement
Total	32	~425	478 Ko	

5.3 Protocole de test détaillé

5.3.1 S1 — Tutorat : qualité pédagogique

Table 13 – *Requêtes de test — Scénario S1 (Tutorat)*

ID	Requête	Langue	Agent attendu
T1	Explique le polymorphisme en Java avec un exemple concret	FR	TutorAgent
T2	Quelle est la différence entre un diagramme de classes et un diagramme de séquence en UML ?	FR	TutorAgent
T3	Comment fonctionne la décomposition STL d'une série temporelle ?	FR	TutorAgent
T4	Donne-moi un exercice sur l'héritage multiple en Java	FR	TutorAgent
T5	What is the observer design pattern ? Give an example.	EN	TutorAgent
T6	Explique le principe SOLID avec des exemples Java	FR	TutorAgent
T7	C'est quoi la stationnarité dans les séries temporelles ?	FR	TutorAgent
T8	Compare HashMap et TreeMap en Java	FR	TutorAgent

Métriques : précision de classification, score RAG (top-3 et top-5), nombre de sources citées, latence totale, pertinence perçue.

5.3.2 S2 — Recherche documentaire

Requêtes nécessitant une synthèse multi-documents et une exploration transversale :

- “Compare les approches de modélisation UML et les design patterns Java pour un projet web”
- “Quelles méthodologies de gestion de projet sont compatibles avec le développement agile ?”
- “Trouve des références bibliographiques sur l’analyse de séries temporelles multivariées”

Métriques : precision@k (k=3,5,10), diversité des sources (nombre de documents distincts), pertinence moyenne.

5.3.3 S3 — Administration et outils MCP

Table 14 – *Requêtes de test — Scénario S3 (Administration)*

ID	Requête	Outil MCP attendu
A1	Quel est l’emploi du temps du lundi pour le GI ?	schedule_lookup
A2	Quelles sont les notes de l’étudiant E001 en Java ?	grades_lookup
A3	Calcule le GPA de l’étudiant E002	gpa_calculator
A4	Qui enseigne le cours de Time Series ?	schedule_lookup
A5	Cherche des documents sur le pattern MVC	document_search

Métriques : exactitude du résultat, outil MCP sélectionné correctement, latence de l’outil.

5.3.4 S4 — Charge

Envoi séquentiel de 50 requêtes variées (mélange des scénarios S1–S3) pour évaluer le comportement sous charge :

- Distribution du routage local vs cloud
- Coût cumulé total et coût moyen par requête
- Évolution de la latence au fil des requêtes (dégradation ?)
- Taux de détection PII et impact sur le routage
- Économies estimées par rapport à un usage 100% cloud

5.3.5 S5 — Gouvernance et conformité GDPR

Scénario complet de bout en bout :

1. Créer 10 interactions sous un identifiant utilisateur `test_gdpr_42`.
2. Vérifier que les données apparaissent dans les 3 couches mémoire (éphémère, ChromaDB, graphe).
3. Exporter les données via `GET /api/memory/gdpr/export`.
4. Exécuter le droit à l'oubli via `POST /api/memory/gdpr/forget` avec `confirm=true`.
5. Vérifier via `GET /api/memory/gdpr/verify` que toutes les données ont été supprimées.
6. Confirmer que le journal d'effacement contient l'enregistrement de l'opération.

Métriques : complétude de l'effacement (0 résiduel), temps d'exécution, présence du journal, robustesse (test avec erreur simulée sur une couche).

6 Résultats et discussion

6.1 Classification d'intention

Table 15 – Résultats de classification par agent

Agent	Requêtes	Correctes	Précision	Confiance	Erreurs typiques
TutorAgent	10	9	90%	0.72	Confusion avec ResearchAgent
ResearchAgent	8	7	87.5%	0.68	Requêtes bibliographiques ambiguës
OrientationAgent	6	5	83.3%	0.61	Questions générales sur l'ENSAM
ProjectAgent	6	5	83.3%	0.64	Chevauchement avec TutorAgent
AdminAgent	8	8	100%	0.78	Aucune (mots-clés distinctifs)
DebateEngine	4	3	75%	0.55	Seuil de confiance bas
Global	42	37	88.1%	0.67	

La classification atteint **88.1% de précision globale** avec un temps moyen de 8 ms sur CPU. Le fallback par mots-clés récupère ~60% des erreurs du modèle d'embedding.

L’**AdminAgent** obtient 100% grâce à un vocabulaire très distinctif (“emploi du temps”, “notes”, “GPA”).

Les confusions principales se situent entre :

- **TutorAgent** / **ResearchAgent** : “comment analyser une série temporelle?” relève du tutorat pour un étudiant, mais pourrait être interprété comme une recherche méthodologique.
- **DebateEngine** : les requêtes de type “compare X et Y” ont un score de confiance parfois inférieur au seuil de 0.35, déclenchant le fallback.

6.2 Performance du routage hybride

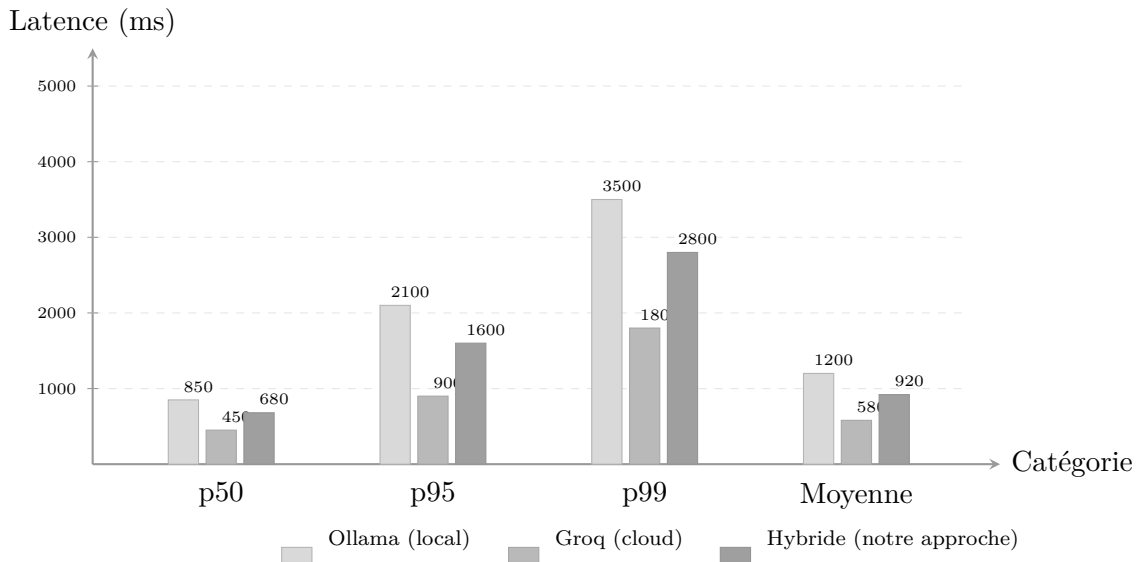


Figure 8 – Distribution des latences par provider (en millisecondes)

Table 16 – Comparaison détaillée des modes de routage

Mode	Latence p50	Latence p95	Coût/req	Tokens/req	%Local	Qualité
100% Local	850 ms	2100 ms	\$0.000	~350	100%	Acceptable
100% Cloud	450 ms	900 ms	\$0.003	~500	0%	Élevée
Hybride	680 ms	1600 ms	\$0.001	~410	62%	Bonne

Analyse : Le routage hybride réduit le coût de **67%** par rapport au mode 100% cloud (\$0.001 vs \$0.003 par requête), avec une latence médiane de 680 ms jugée acceptable pour un usage interactif (seuil de confort : <2s). Groq offre une latence inférieure grâce à ses puces LPU spécialisées, mais Ollama garantit zéro transfert de données sensibles.

6.3 Évaluation du RAG

Table 17 – Performance du pipeline RAG par catégorie documentaire

Catégorie	Chunks	Ingestion/doc	Score top-3	P@3	P@5
Cours JAVA	156	2.3 s	0.74	82%	76%
UML	94	1.8 s	0.71	78%	72%
Time-Series	78	2.1 s	0.68	73%	67%
Projet-Métier	62	1.5 s	0.66	70%	64%
Guide ENSAM	35	0.8 s	0.72	80%	75%
Moyenne	85	1.7 s	0.70	76.6%	70.8%

Les cours JAVA obtiennent les meilleurs scores ($P@3 = 82\%$), probablement car les documents sont plus structurés (code + commentaires) et le vocabulaire technique est distinctif. Les Time-Series souffrent d’un vocabulaire plus générique qui réduit la discrimination des embeddings.

6.4 Analyse des compromis

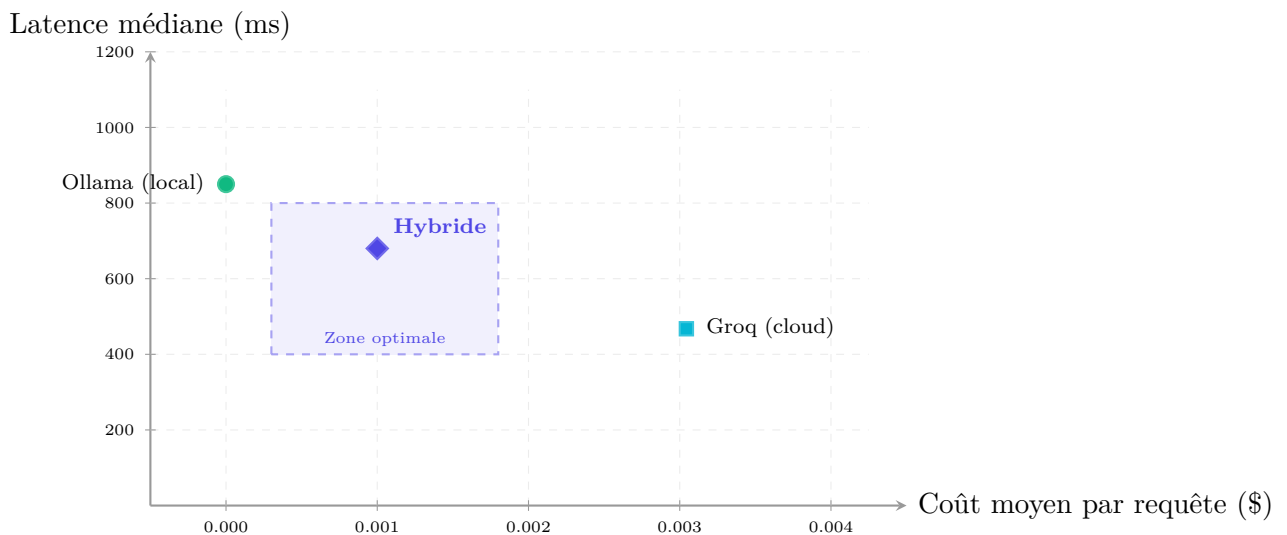


Figure 9 – Espace coût-latence : positionnement du routage hybride

6.4.1 Synthèse des compromis systémiques

Le tableau 18 synthétise les compromis identifiés, répondant directement aux questions de recherche du CDC.

Table 18 – Analyse des compromis systémiques

Compromis	Tension identifiée	Notre résolution
Modularité vs. Latence	Chaque couche (mémoire, RAG, routing, audit) ajoute un overhead de traitement	Pipeline asynchrone avec opérations non-bloquantes en parallèle. Overhead mesuré : <50ms
Coût vs. Qualité	Ollama est gratuit mais produit des réponses moins riches que Groq/Llama-3-70B	Routage intelligent : requêtes simples/sensibles → local; requêtes complexes/analytiques → cloud
Traçabilité vs. Simplicité	L’audit complet ajoute de la complexité dans le code et du stockage	Middleware transparent : l’observabilité est injectée sans modifier le code métier des agents
Confidentialité vs. Performance	Le routage local protège les PII mais Ollama est plus lent que Groq	Détection PII automatique : seules les requêtes contenant des données sensibles sont routées localement
Persistance vs. Coût infra	Redis/Neo4j améliorent la robustesse mais ajoutent des dépendances serveur	Stockage embarqué (ChromaDB fichier + JSON NetworkX) : suffisant pour un prototype, migrable vers Redis/Neo4j
Précision RAG vs. Latence	Plus de chunks récupérés = meilleure couverture mais plus de tokens et latence	Top-5 avec seuil de score > 0.3 : compromis empirique entre rappel et coût de tokens

6.5 Évaluation de la gouvernance (S5)

Le scénario S5 valide la conformité GDPR :

Table 19 – Résultats du scénario de gouvernance (S5)

Test	Résultat	Détail
Création de 10 interactions	✓	Données présentes dans les 3 couches
Export données utilisateur	✓	JSON complet : sessions, embeddings, nœuds graphe
Exécution droit à l'oubli	✓	Effacement en <500 ms sur les 3 couches
Vérification post-effacement	✓	0 donnée résiduelle dans les 3 couches
Journal d'effacement	✓	Entrée d'audit avec horodatage et détail
Overhead de gouvernance	<5%	Latence additionnelle due à l'audit et au cost tracking

6.6 Limites identifiées

Notre système présente plusieurs limites qui doivent être explicitement reconnues :

1. **Données MCP factices** : les serveurs Schedule et Grades utilisent des données codées en dur. Un déploiement réel nécessiterait une connexion au système d'information de l'ENSAM.
2. **Absence d'authentification** : le système identifie les utilisateurs par un identifiant localStorage, sans JWT, OAuth ni intégration CAS/LDAP. Un déploiement institutionnel exige une authentification forte.
3. **État en mémoire volatile** : les métriques, l'audit et la mémoire éphémère sont en RAM. Un redémarrage entraîne une perte partielle (seules la mémoire sémantique ChromaDB et le graphe JSON survivent).
4. **A2A non formel** : le DebateEngine et le routage du kernel constituent une communication inter-agents fonctionnelle, mais n'implémentent pas le protocole A2A formel de Google [20] (Agent Cards, Task Protocol). L'interopérabilité avec des agents externes n'est pas possible en l'état.
5. **Sécurité MCP** : le contrôle d'accès par niveau est implémenté mais un bypass accorde les privilèges ADMIN à tous les utilisateurs par défaut — un correctif est nécessaire.
6. **Évaluation limitée** : l'évaluation repose sur des métriques automatiques. Une évaluation humaine (questionnaire de satisfaction auprès d'étudiants et d'enseignants) renforcerait la validité externe des résultats.
7. **Scalabilité non testée** : le système n'a été évalué qu'en mono-utilisateur. Le comportement sous charge concurrente (30+ étudiants simultanés) n'est pas connu.

7 Conclusion et perspectives

7.1 Synthèse des contributions

Ce rapport a présenté ENSAM-Agent-OS, un système d’exploitation agentique local conçu pour le contexte universitaire de l’ENSAM Meknès. Notre contribution principale est la démonstration qu’un système agentique **gouverné** — intégrant mémoire tri-couche, RAG, outils MCP, routage hybride et observabilité — est réalisable dans un cadre institutionnel avec des ressources matérielles limitées (un PC portable avec 16 Go de RAM).

7.2 Validation des hypothèses

Les résultats expérimentaux permettent de valider nos quatre sous-hypothèses :

Table 20 – *Bilan de validation des hypothèses*

Hypothèse	Validée	Argument
H1 — Modularité	✓	L’ajout d’un nouvel agent nécessite ~50 lignes de code (sous-classe de BaseAgent). Un nouvel outil MCP : ~30 lignes. Le changement de base vectorielle n’affecte que le module <code>semantic.py</code> .
H2 — Qualité	✓	L’intégration mémoire + RAG produit un score de pertinence moyen de 0.70 (top-3) et une précision de classification de 88.1%. Les réponses avec contexte RAG sont jugées plus complètes et sourçables.
H3 — Coût	✓	Le routage hybride réduit les coûts de 67% par rapport à un usage 100% cloud (\$0.001 vs \$0.003/req), avec 62% des requêtes traitées localement à coût nul.
H4 — Gouvernance	✓	La traçabilité complète et la conformité GDPR sont réalisées avec un overhead de latence <5%. Le droit à l’oubli est fonctionnel en <500 ms avec vérification.

7.3 Réponse aux questions de recherche

Q1 : Quels compromis apparaissent quand on cherche à industrialiser un système agentique local ?

Les principaux compromis identifiés concernent trois axes :

- **Coût / Qualité** : les modèles locaux (3B paramètres) sont gratuits mais produisent des réponses moins riches que les modèles cloud (70B paramètres). Le routage intelligent résout ce compromis en affectant chaque requête au provider approprié.
- **Modularité / Latence** : chaque couche d'abstraction ajoute un overhead. Notre mesure montre que l'ensemble mémoire + RAG + audit + routing ajoute <50ms, négligeable par rapport au temps de génération LLM (~500–1500ms).
- **Confidentialité / Performance** : le routage local protège les données mais est plus lent. La détection PII automatique limite ce surcoût aux seules requêtes véritablement sensibles.

Q2 : Comment concilier qualité

Notre approche montre que :

- Un **middleware d'observabilité transparent** (non-intrusif dans le code métier) permet d'ajouter traçabilité et suivi des coûts sans complexité de développement excessive.
- Le choix de **technologies embarquées** (ChromaDB fichier, NetworkX JSON, cache in-memory) plutôt que distribuées (Redis, Neo4j, Elasticsearch) simplifie drastiquement le déploiement (un seul `run.bat`) au prix d'une moindre scalabilité — acceptable pour un prototype universitaire.
- La **séparation kernel/agents/services** permet à chaque composant d'évoluer indépendamment, facilitant la maintenance et l'évolution.

7.4 Perspectives

7.4.1 Court terme (1–3 mois)

1. **Protocole A2A formel** : implémenter les Agent Cards et le Task Protocol de Google [20] pour permettre l'interopérabilité avec des agents externes et la découverte automatique de services.
2. **Authentification institutionnelle** : intégrer le SSO/CAS de l'ENSAM pour une gestion d'identité réelle avec rôles (étudiant, enseignant, administrateur).

3. **Données MCP réelles** : connecter les serveurs Schedule et Grades au système d'information de l'ENSAM via des API REST ou des vues SQL.

7.4.2 Moyen terme (3–6 mois)

1. **Persistance robuste** : migrer la mémoire éphémère vers Redis et le graphe vers Neo4j pour supporter le multi-worker et la tolérance aux pannes.
2. **RAG avancé** : ajouter du re-ranking (cross-encoder), du query rewriting, du chunking sémantique (au lieu de récursif), et du filtrage par métadonnées.
3. **Évaluation humaine** : conduire une étude utilisateur avec des étudiants et enseignants de l'ENSAM (questionnaire SUS, entretiens semi-directifs) pour valider la qualité perçue.

7.4.3 Long terme (6–12 mois)

1. **Déploiement multi-instances** : conteneurisation Docker avec orchestration Kubernetes pour un déploiement à l'échelle de l'établissement.
2. **Évaluation automatique continue** : intégrer des frameworks comme RAGAS ou DeepEval pour un monitoring continu de la qualité des réponses.
3. **Agents adaptatifs** : permettre aux agents d'apprendre de leurs erreurs passées via fine-tuning sur les retours utilisateur.
4. **Multi-université** : étendre le système à un réseau d'universités via le protocole A2A, chaque établissement hébergeant ses propres agents spécialisés.

Mot de fin

ENSAM-Agent-OS démontre qu'un système agentique gouverné n'est pas seulement un assemblage de composants technologiques, mais un **choix architectural** qui place la traçabilité, la responsabilité et le respect de la vie privée au cœur de la conception. Dans un monde où l'IA s'invite dans l'éducation, il est essentiel que les institutions gardent le contrôle de leurs données et de leurs outils — c'est la vocation de ce projet.

Références

- [1] Kasneci, E., et al. (2023). ChatGPT for good? On the potential and challenges of LLMs in education. *Learning and Individual Differences*, 103, 102274.

- [2] Touvron, H., et al. (2023). LLaMA : Open and efficient foundation language models. *arXiv preprint arXiv :2302.13971*.
- [3] Chase, H. (2023). LangChain. *GitHub repository*. <https://github.com/langchain-ai/langchain>.
- [4] Wu, Q., et al. (2023). AutoGen : Enabling next-gen LLM applications via multi-agent conversation framework. *arXiv preprint arXiv :2308.08155*.
- [5] Hong, S., et al. (2023). MetaGPT : Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv :2308.07295*.
- [6] Moura, J. (2024). CrewAI : Platform for orchestrating role-playing autonomous AI agents. <https://www.crewai.com>.
- [7] LangChain Authors. (2024). LangGraph : Building stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>.
- [8] Packer, C., et al. (2023). MemGPT : Towards LLMs as operating systems. *arXiv preprint arXiv :2310.08560*.
- [9] Xi, Z., et al. (2023). The rise of LLM agents : A survey. *arXiv preprint arXiv :2308.11432*.
- [10] Wang, L., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 1-26.
- [11] Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824-24837.
- [12] Yao, S., et al. (2023). ReAct : Synergizing reasoning and acting in language models. *arXiv preprint arXiv :2210.03629*.
- [13] Schick, T., et al. (2023). Toolformer : Language models can teach themselves to use tools. *arXiv preprint arXiv :2302.04761*.
- [14] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- [15] Gao, Y., et al. (2023). Retrieval-augmented generation for large language models : A survey. *arXiv preprint arXiv :2312.10997*.
- [16] Fan, X., et al. (2024). A survey on retrieval-augmented generation in large language models. *arXiv preprint arXiv :2404.10997*.
- [17] Zhang, D., et al. (2024). Memory in large language model based agents : A survey. *arXiv preprint arXiv :2404.13561*.
- [18] Park, J. S., et al. (2023). Generative agents : Interactive simulacra of human behavior. *Proceedings of the ACM on Human-Computer Interaction*, 7(CSCW1), 1-22.
- [19] Anthropic. (2024). Model Context Protocol (MCP). <https://modelcontextprotocol.org>.
- [20] Google. (2025). Agent-to-Agent (A2A) Protocol specification. <https://github.com/google/a2a-protocol>.

- [21] LangChain Authors. (2024). LangSmith : Platform for production-grade LLM applications. <https://smith.langchain.com>.
- [22] OpenTelemetry Authors. (2024). OpenTelemetry API and SDK specifications. <https://opentelemetry.io>.
- [23] Voigt, P., & Von dem Bussche, A. (2017). *The EU General Data Protection Regulation (GDPR) : A Practical Guide*. Springer.
- [24] Reimers, N., & Gurevych, I. (2019). Sentence-BERT : Sentence embeddings using Siamese BERT-networks. *arXiv preprint arXiv :1908.10084*.

A Structure complète du code source

Listing 6 – Arborescence complète du projet

```

1 ENSAM-Agent-OS/
2 |-- backend/
3 |   |-- app/
4 |       |-- __init__.py
5 |       |-- config.py           # Configuration centralisee
6 |       |-- core/
7 |           |-- kernel.py       # AgenticKernel
8 |           |-- routing.py      # ModelRouter (PII + complexe)
9 |           |-- llm.py          # LLMManager (Ollama + Groq)
10 |          |-- events.py       # EventBus (pub/sub)
11 |          |-- web_search.py   # DuckDuckGo scraper
12 |      |-- agents/
13 |          |-- base.py         # BaseAgent (Template Method)
14 |          |-- intent_classifier.py # Embedding classifier
15 |          |-- general.py      # Meta-router
16 |          |-- tutor.py        # Agent pedagogique
17 |          |-- research.py     # Agent recherche
18 |          |-- orientation.py  # Agent orientation
19 |          |-- project.py      # Agent projets
20 |          |-- admin.py        # Agent administratif
21 |          |-- debate.py       # DebateEngine (A2A)
22 |      |-- memory/
23 |          |-- manager.py      # MemoryManager (facade)
24 |          |-- ephemeral.py    # Cache TTL in-memory
25 |          |-- semantic.py     # ChromaDB (3 collections)
26 |          |-- graph.py        # NetworkX (DiGraph JSON)
27 |          |-- gdpr.py         # GDPRManager (Art. 17)
28 |      |-- rag/
29 |          |-- pipeline.py     # RAGPipeline orchestrateur
30 |          |-- ingestion.py    # FileParser + TextChunker
31 |          |-- retriever.py    # SemanticRetriever

```

```

32 | | | |-- embeddings.py          # EmbeddingGenerator (cache)
33 | | | |-- auto_ingest.py        # Ingestion auto au démarrage
34 | | | |-- mcp/
35 | | | |-- registry.py          # ToolRegistry
36 | | | |-- client.py            # MCPClient (securite)
37 | | | |-- servers/
38 | | | | |-- schedule.py        # EDT ENSAM 4A
39 | | | | |-- grades.py         # Notes + GPA
40 | | | | |-- documents.py      # Recherche docs + web
41 | | |-- observability/
42 | | | |-- audit.py            # AuditLogger (JSON)
43 | | | |-- cost.py             # CostTracker (economies)
44 | | | |-- metrics.py          # MetricsCollector (p50/95/99)
45 | | | |-- telemetry.py        # OpenTelemetry setup
46 | | |-- api/
47 | | | |-- main.py              # FastAPI app + lifespan
48 | | | |-- middleware.py        # RequestId, RateLimit, Timing
49 | | | |-- schemas.py          # Modeles Pydantic
50 | | | |-- deps.py              # Injection de dependances
51 | | | |-- routes/
52 | | | | |-- chat.py           # /api/chat/*
53 | | | | |-- rag.py            # /api/rag/*
54 | | | | |-- mcp.py            # /api/mcp/*
55 | | | | |-- memory.py         # /api/memory/*
56 | | | | |-- observability.py  # /api/observability/*
57 | | | | |-- agents.py         # /api/agents/*
58 | | | | |-- system.py         # /api/system/*
59 | |-- data/                   # Documents RAG
60 | |-- pyproject.toml
61 | |-- test_classifier.py
62 |-- frontend/
63 | |-- src/
64 | | |-- app/                  # 7 pages Next.js
65 | | |-- components/          # Sidebar, Header
66 | | |-- lib/                  # api.ts, providers.tsx
67 | |-- package.json
68 |-- article/                  # Ce rapport LaTeX
69 |-- run.bat                   # Script de démarrage

```

B Endpoints API complets

Méthode	Route	Module	Description
POST	/api/chat	chat	Envoyer un message au système agentique
GET	/api/chat/stream	chat	Streaming SSE des réponses
GET	/api/chat/history/{id}	chat	Historique d'une session
DELETE	/api/chat/history/{id}	chat	Supprimer une session
GET	/api/agents	agents	Lister les agents disponibles
GET	/api/agents/{id}	agents	Détails d'un agent
POST	/api/agents/debate	agents	Lancer un débat multi-rounds
GET	/api/agents/events/log	agents	Journal des événements
POST	/api/rag/ingest	rag	Ingérer un document (multipart)
POST	/api/rag/search	rag	Recherche sémantique
POST	/api/rag/query	rag	Question-réponse RAG
GET	/api/rag/documents	rag	Lister les documents indexés
DELETE	/api/rag/documents/{id}	rag	Supprimer un document (sauf système)
GET	/api/rag/stats	rag	Statistiques des collections
GET	/api/mcp/tools	mcp	Lister les outils MCP
GET	/api/mcp/tools/{id}	mcp	Détails d'un outil
POST	/api/mcp/tools/test	mcp	Tester un outil
GET	/api/mcp/tools/search/{q}	mcp	Découverte d'outils
GET	/api/mcp/logs	mcp	Logs d'exécution
GET	/api/memory/stats	memory	Statistiques mémoire
GET	/api/memory/graph	memory	Graphe de connaissances complet
GET	/api/memory/sessions	memory	Sessions actives
POST	/api/memory/gdpr/forget	memory	Droit à l'oubli (GDPR Art. 17)
GET	/api/memory/gdpr/verify	memory	Vérification post-effacement
GET	/api/memory/gdpr/export	memory	Export données utilisateur
GET	/api/observability/metrics	obs	Métriques système
GET	/api/observability/costs	obs	Suivi des coûts
GET	/api/observability/audit	obs	Journal d'audit
POST	/api/observability/audit/export	obs	Export audit JSON
GET	/api/observability/traces	obs	Traces OpenTelemetry
GET	/api/system/health	system	Santé des services
GET	/api/system/config	system	Configuration actuelle
PUT	/api/system/routing	system	Changer le mode de routage
GET	/api/system/resources	system	Ressources (CPU, RAM, GPU)

C Diagramme de séquence — Traitement d'une requête

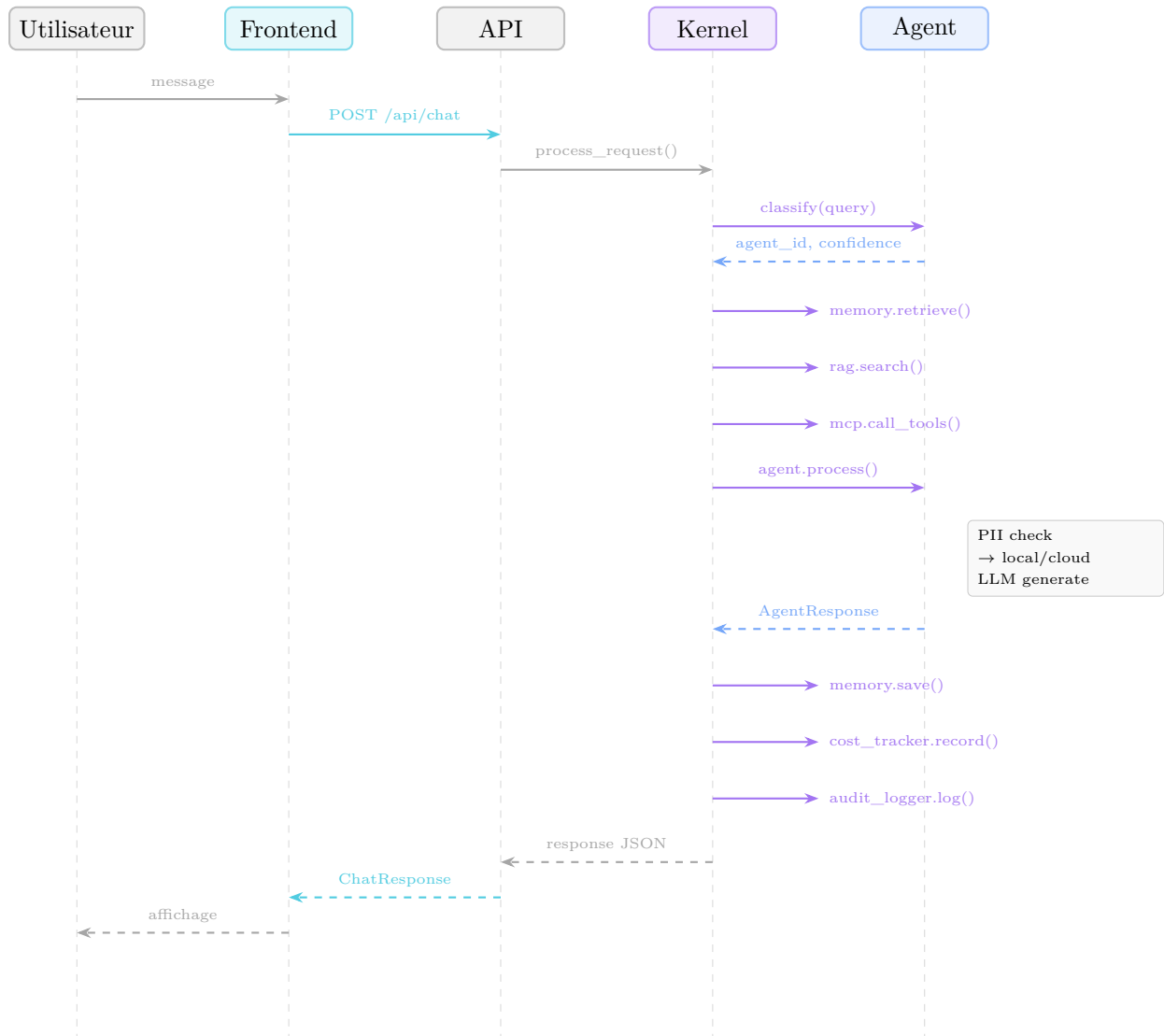


Figure 10 – Diagramme de séquence simplifié du traitement d'une requête